

Aula 2 — Setup, Manual Estruturado & Unit Testing

"Antes de automatizar UI, e o que vem antes?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 28/05/2026

Abertura — Fluxo Fork + PR (~15min)

Antes do conteúdo técnico, **abrimos o deck de entrega via GitHub:**

 `compartilhado/slides-source/intro-fluxo-pr-github.pdf`

 Repo público: `slides/intro-fluxo-pr-github.pdf`

Cobre: fork → clone → branch → commit → push → PR → link Canvas.

Termina com **demo ao vivo** num repo dummy.

| Padrão de toda Aula 2: garantir que ninguém trava na entrega da próxima atividade.

Recap da Aula 1

- Ecosystema mobile: 3M apps, 77% abandono
- Pirâmide de Knott vs Cohn
- Tipos de teste mobile (10 categorias)
- Real device vs emulator vs cloud farm

Atividade 1 — Análise de Cobertura entregue?

Agenda da aula (3h30)

1. **Bloco 1 (90min)** — Setup completo + Manual estruturado (SBTM)
2. **Intervalo (15min)**
3. **Bloco 2 (90min)** — Hands-on: testes unitários iOS + Android + RN
4. **Wrap-up (15min)** — Atividade 2

Objetivos de aprendizagem

- **Configurar** ambiente Xcode + Android Studio + ADB tools
- **Estruturar** testes manuais com SBTM (Bach)
- **Aplicar** heurísticas FEW HICCUPPS (Bolton)
- **Implementar** testes unitários em XCTest, JUnit + MockK, Jest + RNTL
- **Aplicar** test doubles (Meszaros) corretamente

Setup ambiente — checklist

```
# Node 22 LTS
nvm install 22 && nvm use 22

# Watchman + Cocoapods
brew install watchman cocoapods

# Maestro CLI
curl -Ls "https://get.maestro.mobile.dev" | bash

# Android Studio Iguana+ + AVD
# Xcode 16+

# Verificar
xcodebuild -version      # 16.x
adb --version            # 35.x+
maestro --version        # 1.x+

# RN starter
npx react-native@latest init MeuApp
```

Manual testing estruturado — por que importa

Em mobile, **manual ainda é peça central** porque automação não cobre:

- Sensação de UX (responsividade tátil)
- Comportamento em conectividade variável
- Interação com sensores
- Permissões iOS/Android dialogs
- Comportamento em background/foreground transitions
- Acessibilidade real

Mas manual sem estrutura é caos. SBTM resolve.

Session-Based Test Management (SBTM)

Proposto por **James Bach**.

Sessão: unidade de tempo dedicada (60–120min).

Estrutura:

1. **Charter** — missão da sessão em 1 frase
2. **Tester** — quem executa
3. **Duração** — alvo (60min, 90min, 120min)
4. **Notas** — log do que fez, o que viu
5. **Bugs** — issues levantadas
6. **Issues** — perguntas/debates
7. **Debriefing** — discussão pós-sessão

Estrutura sem virar script. Foco com flexibilidade.

Charter — exemplo

CHARTER:

Explorar fluxo de checkout do app de e-commerce
focando em: cupons, métodos de pagamento alternativos
e cancelamentos antes de confirmação.

Objetivo: descobrir bugs de UX/funcionais.

Duração: 90min.

| Charter dá foco. Tester explora dentro do escopo, livre pra perseguir descobertas.

Heurísticas — FEW HICCUPPS

Michael Bolton. Heurística pra cobertura sistemática:

- **Familiar** — conformidade com padrões da plataforma
- **Explicit** — comportamento explicitamente especificado
- **World** — comportamento esperado pelo usuário real
- **History** — comportamento histórico do produto
- **Image** — imagem da empresa/produto
- **Comparable products** — comparáveis no mercado
- **Claims** — promessas marketing/docs
- **User desires** — desejos não declarados
- **Purpose** — propósito do produto
- **Product** — coerência interna
- **Statutes** — leis, regulações

Whittaker tours

James Whittaker propõe "tours" pelo app:

- **Money tour** — onde o produto gera receita
- **Landmark tour** — features principais
- **Antisocial tour** — entradas inválidas, edge cases
- **Obsessive tour** — repetir mesma ação ad infinitum
- **All-nighter tour** — deixar app aberto 24h+
- **FedEx tour** — como dado entra e sai do app

Use como prompt para sessões SBTM.

Bug reporting — template eficaz

TÍTULO: [iOS 17] Botão Comprar não responde após cupom expirado

PASSOS:

1. Adicionar item ao carrinho
2. Aplicar cupom expirado (CUPOM_TESTE_2024)
3. Tap no botão "Comprar"

ESPERADO: Mensagem "Cupom expirado, remova para continuar"

ATUAL: Botão não responde, sem feedback visual

REPRO RATE: 5/5 (sempre)

DEVICE: iPhone 14 Pro, iOS 17.5

APP VERSION: 2.3.1 (build 2310)

SCREENCAST: link.loom.com/abc123

Repro rate é diferencial. "Aconteceu uma vez" é hipótese, não bug.

Testes unitários iOS — XCTest

```
import XCTest
@testable import MyApp

final class CartTests: XCTestCase {
    var sut: ShoppingCart!

    override func setUp() {
        super.setUp()
        sut = ShoppingCart()
    }

    override func tearDown() {
        sut = nil
        super.tearDown()
    }

    func test_addItem_increasesItemCount() {
        let item = Item(id: "1", name: "Café", price: 5.00)
        sut.add(item: item)
        XCTAssertEqual(sut.itemCount, 1)
    }

    func test_total_returnsSumOfItemPrices() {
        sut.add(item: Item(id: "1", name: "A", price: 10))
        sut.add(item: Item(id: "2", name: "B", price: 20))
        XCTAssertEqual(sut.total, 30, accuracy: 0.01)
    }
}
```

XCTest — Quick & Nimble (BDD)

```
import Quick
import Nimble
@testable import MyApp

class CartSpec: QuickSpec {
    override func spec() {
        describe("ShoppingCart") {
            var cart: ShoppingCart!

            beforeEach { cart = ShoppingCart() }

            context("when adding an item") {
                it("increases the item count") {
                    cart.add(item: Item(id: "1", name: "Café", price: 5))
                    expect(cart.itemCount).to(equal(1))
                }

                it("includes the item in the total") {
                    cart.add(item: Item(id: "1", name: "Café", price: 5))
                    expect(cart.total).to(beCloseTo(5.00, within: 0.01))
                }
            }
        }
    }
}
```

Testes unitários Android — JUnit 5 + MockK

```
import io.mockk.*
import org.junit.jupiter.api.*
import kotlin.test.assertEquals

class CheckoutUseCaseTest {

    private lateinit var paymentGateway: PaymentGateway
    private lateinit var sut: CheckoutUseCase

    @BeforeEach
    fun setup() {
        paymentGateway = mockk()
        sut = CheckoutUseCase(paymentGateway)
    }

    @Test
    fun `executes payment when cart total is positive`() = runTest {
        coEvery { paymentGateway.charge(any()) } returns PaymentResult.Success("tx-1")

        val result = sut.execute(Cart(total = 100.0))

        assertEquals("tx-1", result.transactionId)
        coVerify(exactly = 1) { paymentGateway.charge(100.0) }
    }
}
```

Robolectric — quando usar

Roda testes que dependem do framework Android **na JVM**, sem emulator.

```
@RunWith(RobolectricTestRunner::class)
@Config(sdk = [34])
class SettingsViewModelTest {

    @Test
    fun `reads stored theme from SharedPreferences`() {
        val context = ApplicationProvider.getApplicationContext<Context>()
        val prefs = context.getSharedPreferences("app", Context.MODE_PRIVATE)
        prefs.edit().putString("theme", "dark").commit()

        val sut = SettingsViewModel(context)

        assertEquals("dark", sut.currentTheme)
    }
}
```

Útil para testes que dependem de Context/Resources/SharedPreferences sem boot de emulator.

Testes unitários RN — Jest + RNTL

```
import { render, fireEvent, waitFor } from '@testing-library/react-native';
import LoginScreen from '../LoginScreen';

describe('LoginScreen', () => {
  it('disables submit when email is invalid', () => {
    const { getByPlaceholderText, getByText } = render(<LoginScreen />);

    fireEvent.changeText(getByPlaceholderText('Email'), 'invalid');

    expect(getByText('Entrar')).toBeDisabled();
  });

  it('calls onSubmit with valid credentials', async () => {
    const onSubmit = jest.fn();
    const { getByPlaceholderText, getByText } = render(<LoginScreen onSubmit={onSubmit} />);

    fireEvent.changeText(getByPlaceholderText('Email'), 'a@b.com');
    fireEvent.changeText(getByPlaceholderText('Senha'), 'secret');
    fireEvent.press(getByText('Entrar'));

    await waitFor(() => {
      expect(onSubmit).toHaveBeenCalledWith({ email: 'a@b.com', password: 'secret' });
    });
  });
});
```

Test doubles (Meszaros)

Tipo	O que faz	Quando usar
Dummy	Objeto qualquer, nunca usado	Preenche parâmetro
Stub	Retorna respostas pré-programadas	State-based testing
Spy	Stub + grava chamadas	Verificar quantas vezes algo foi chamado
Mock	Verifica chamadas com expectativas	Interaction-based testing
Fake	Implementação simplificada (in-memory DB)	Replicar dependência cara

Não confunda. Stub responde, mock observa.

Cobertura — JaCoCo, xccov, Istanbul

```
# Android (JaCoCo)
./gradlew jacocoTestReport
open app/build/reports/jacoco/jacocoTestReport/html/index.html

# iOS (xccov)
xcodebuild test -scheme MyApp -enableCodeCoverage YES
xcrun xccov view --report TestResults.xcresult

# RN (Jest + Istanbul)
jest --coverage
open coverage/lcov-report/index.html
```

| Target inicial: **70%+ em data/domain layer**. UI pode ter cobertura menor.

Cuidado com cobertura inflada

✗ **Test que renderiza componente mas não tem assertion** → coverage 100%, valor 0.

✓ **Cobertura útil = código executado e verificado.**

| Mutation testing (Stryker) prova isso. Veremos na M3 (QA EAD) e na Aula 6.

Hands-on — vamos fazer juntos (90min)

Roteiro

1. Clone starter `aula-02` (RN + Android + iOS) (5min)
2. Configurar Jest + RNTL no RN (10min)
3. Escrever 3 unit tests em domain layer (15min)
4. JUnit + MockK no Android (15min)
5. XCTest no iOS (15min)
6. Cobertura $\geq 70\%$ em 1 layer (15min)
7. Sessão SBTM aplicada ao app (15min)

Pitfalls comuns

- ✗ **Sleep em unit test** — anti-pattern absoluto
- ✗ **Mock de tudo** — testes acoplam à implementação, não ao comportamento
- ✗ **Test names sem nada explicativo** (`testFunc1`) — rename pra revelar intenção
- ✗ **Coverage como métrica única** — alta cobertura com mutation score baixo é ilusão
- ✗ **Manual testing sem charter** — vira clicar aleatório

Atividade 2: Setup + Suíte Unitária (10 pts)

Entrega: até 07/06.

Entregar:

1. **Setup iOS funcional** (Xcode + simulator + tests rodando)
2. **Setup Android funcional** (Studio + emulator + tests rodando)
3. **Cobertura $\geq 70\%$** em data layer
4. **Cobertura $\geq 70\%$** em domain layer
5. **Critério eliminatório:** repo deve clonar e rodar em $< 15\text{min}$

Leituras obrigatórias (pré-Aula 3)

- Knott — *Hands-On Mobile App Testing*, cap 5–6
- Meszaros, G. — *xUnit Test Patterns* (Addison-Wesley 2007), cap 5
- Khorikov, V. — *Unit Testing: Principles, Practices, and Patterns* (Manning 2020), cap 4
- Bach, J. — *Session-Based Test Management* (artigo seminal)

Próxima aula (08/06)

Aula 3 — Espresso, XCUITest e Detox

Pré-requisito:

- Atividade 2 entregue
- Apps starter buildando em iOS + Android

Obrigado!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Próxima segunda, 08/06, 19:00